

Unit 2 — Agent Architectures, Multi-Agent Systems & Tool Use

Autonomy & characteristics • Deliberative / Reactive / Hybrid architectures • MAS & interaction mechanisms • Tool & function calling • Planning algorithms (intro)

Contents

1. Intelligent-agent characteristics (recap + advanced)
2. Architectures of agentic systems
3. Deliberative architecture
4. Reactive architecture
5. Hybrid architecture
6. Multi-agent systems (MAS)
7. Characteristics of MAS
8. Interaction mechanisms: cooperation, coordination, negotiation
9. MAS algorithm for travel booking
10. Tool use in agents
11. Tool calling vs function calling
12. Defining tools (docstring / JSON schema)
13. Types of tools & key considerations
14. CrewAI tool example & significance of tools
15. Planning algorithms — overview & less-practical algorithms
16. Q&A, summary

1. Intelligent Agent Characteristics

Intelligent agent: Complex, self-governed entity that perceives its environment and acts to achieve goals; from rule-following systems to systems that learn and adapt.

Characteristic	Meaning
Reactivity	Responds in real time to changes/events in the environment; continuously monitors and adjusts → actions stay relevant.
Proactiveness	Anticipates needs/challenges/opportunities, takes initiative, goal-oriented; not just reacting.
Social ability	Communication, coordination and negotiation with other agents/humans in multi-agent systems; leverages collective intelligence.
Learning & adaptation	Learns from experience (machine learning, reinforcement learning, evolutionary algorithms), refines decision-making and improves performance.
Reasoning & planning	Analyses complex situations, formulates strategies via knowledge representation, logical inference and planning algorithms.
Autonomy & self-governance	Makes decisions/takes actions independently without constant human supervision — vital where continuous human control is impractical or impossible.

Applications: robotics, decision-support systems, virtual assistants, gaming, simulations.

2. Architectures of Agentic Systems

Agentic systems executing complex goals autonomously use architectural patterns that define the structure and behaviour for **perceiving, reasoning, learning and acting**. Three main patterns: **deliberative, reactive,**

hybrid.

2.1 Deliberative architecture (knowledge-based / symbolic)

Relies on **explicit representations of knowledge and reasoning** to reach decisions; follows the **sense → plan → act** cycle.

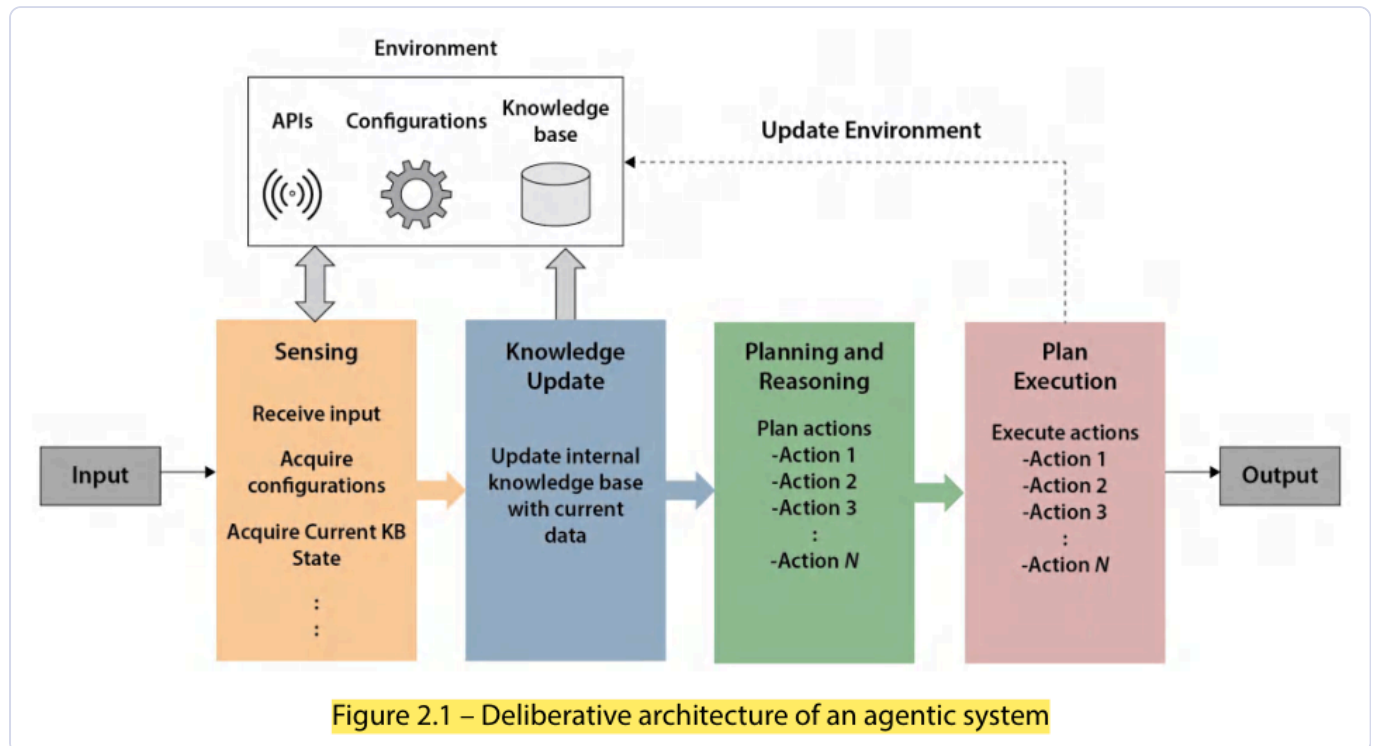


Figure 2.1 — Deliberative architecture of an agentic system

Sensing → Knowledge updating → Planning & reasoning → Plan execution

1. **Sensing** — perceive/acquire environment information through sensors/inputs.
 2. **Knowledge updating** — update internal knowledge base to keep an accurate representation of the current state.
 3. **Planning and reasoning** — formulate plans/decisions via constraint satisfaction, logical inference, search algorithms, heuristic-based planning.
 4. **Plan execution** — carry out actions, possibly modifying the environment or achieving goals.
- **Key component:** a **knowledge base** storing symbolic representations of environment, goals, constraints and domain knowledge, encoded in formal language/logic to allow inference.
 - **Techniques:** rule-based reasoning, constraint satisfaction, heuristic search.
 - **Advantages:** excels at complex reasoning, planning, problem-solving, decision-making in *well-defined environments*; handles uncertainty/ambiguity via probabilistic reasoning, fuzzy logic, belief revision.
 - **Disadvantages:** **computational cost** of maintaining/reasoning over big knowledge bases (limits real-time responsiveness); knowledge hard to formalise or constantly evolving.
 - Often combined with reactive components → hybrid. Used in robotics, decision-support systems, intelligent tutoring systems.

2.2 Reactive architecture (behaviour-based / stimulus-response)

Provides **immediate responses to stimuli**. No explicit world model or complex reasoning — **directly maps perceptions onto actions** using simple condition-action rules or neural networks.

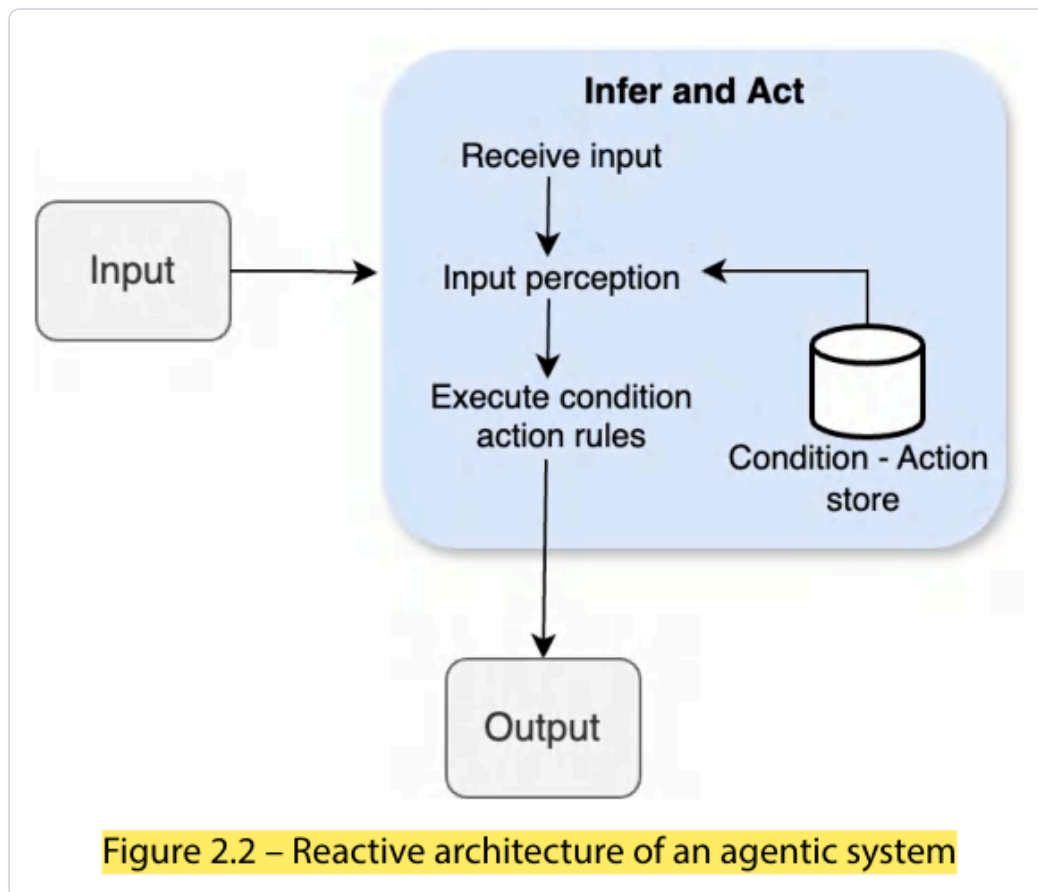


Figure 2.2 — Reactive architecture (input → perception → condition-action rules → output)

Strengths	Limitations
Speed & responsiveness — coupling perception to action bypasses slow reasoning. Robustness & fault tolerance — less susceptible to noise/incomplete information; simple standalone components. Handling uncertainty — reacts to current situation without a precise world model. Parallel & distributed processing — multiple reactive modules run simultaneously and independently → scalability, modularity. Emergence of complex behaviour from interaction of simple behaviours.	Lack of long-term planning — focuses on immediate responses; hard to pursue multi-step goals. Limited reasoning & abstraction — struggles with abstract reasoning, generalisation, symbolic manipulation. Limited learning — fixed rules/behaviours, little adaptation over time.

Widely used where real-time responsiveness, robustness and uncertainty handling matter: **robotics, AI video games, control systems**; also serve as components inside hybrid architectures.

2.3 Hybrid architecture

Combines strengths of deliberative and reactive designs in a **layered structure**:

- **Reactive layer** — fast, low-level responses; real-time interaction; responsive, fault-tolerant, handles uncertainty.
- **Deliberative layer** — high-level reasoning/planning; richer representation of environment, goals, constraints; long-term plans and abstract reasoning.
- **Interaction**: reactive layer gives real-time feedback/situational awareness upward; deliberative layer guides it with high-level plans, goals and constraints.

Techniques used in hybrid architectures:

- **Task decomposition** — reactive layer handles low-level, time-critical tasks; deliberative layer does planning/coordination.
- **Multiplan selection** — deliberative layer generates several plans; reactive layer dynamically selects the best for current conditions.
- **Planning with external modules** — path planning, resource allocation, scheduling, domain-specific algorithms.
- **Reflection and refinement** — learn from outcomes and refine reasoning/planning.
- **Memory-augmented planning** — keep history of past experiences/decisions for future planning.

Architecture	Best when
Deliberative	Intricate reasoning & decision-making in well-defined scenarios
Reactive	Dynamic environments needing swift, adaptive responses
Hybrid	Balance of responsiveness and reasoning; more capable, adaptable systems (active research area)

Choosing a pattern Depends on **task complexity, environmental uncertainty and need for real-time responsiveness**.

3. Multi-Agent Systems (MAS)

Definition A subfield of **distributed AI**: several intelligent agents that **interact, cooperate and coordinate** to execute tasks and achieve collective goals. Each agent is typically autonomous, perceives via sensors, has a reasoning mechanism, and acts on decisions. A MAS may contain software programs, robots or humans with specialised capabilities.

Application domains

Domain	Role of MAS
Supply chain management & logistics	Agents for suppliers, manufacturers, distributors, retailers decide locally and collaborate on resource allocation, inventory, transport planning.
Traffic control & transportation	Agents = vehicles, traffic lights, management centres; reduce congestion, coordinate signals, find optimal routes.
Robotics & manufacturing	Coordinate multiple robots (assembly, welding, material handling) for synchronized operation.
Environmental monitoring & resource management	Water distribution, forestry, wildlife conservation; agents = stakeholders, sensors, decision makers.
Distributed sensor networks	Each node an agent; local processing + information fusion (surveillance, disaster response).
Intelligent virtual environments & simulations	Realistic simulation of social systems, economic models, military operations.

Key advantages: distributes problem-solving, leverages collective intelligence and specialisation, robustness/fault tolerance via decentralised decisions, integrates heterogeneous components, flexible & scalable — better than monolithic centralised approaches.

3.1 Characteristics of MAS

Characteristic	Description
Autonomy	Each agent self-governing, decides from its perception and objectives; no centralised control.
Interaction	Communicate via defined protocols: share information, negotiate tasks, coordinate actions — cooperation, coordination or competition.
Adaptability	Change behaviour when environment or agents' goals change.
Distributed control	Decision-making spread over agents; failure of one agent doesn't necessarily affect the whole system.
Scalability	Agents can be added/removed as needed.
Heterogeneity	Agents may have different architectures, capabilities, goals.
Decentralised data & knowledge	Knowledge distributed among agents → no single point of failure; local knowledge-based operation.

3.2 Interaction mechanisms in MAS

(a) Cooperation

Agents work together toward a **common goal**; important when no single agent can achieve the objective alone. Agents pool knowledge, resources and efforts — divide tasks, combine expertise, complement abilities. *Example: disaster rescue with drones, robots and humans locating victims.*

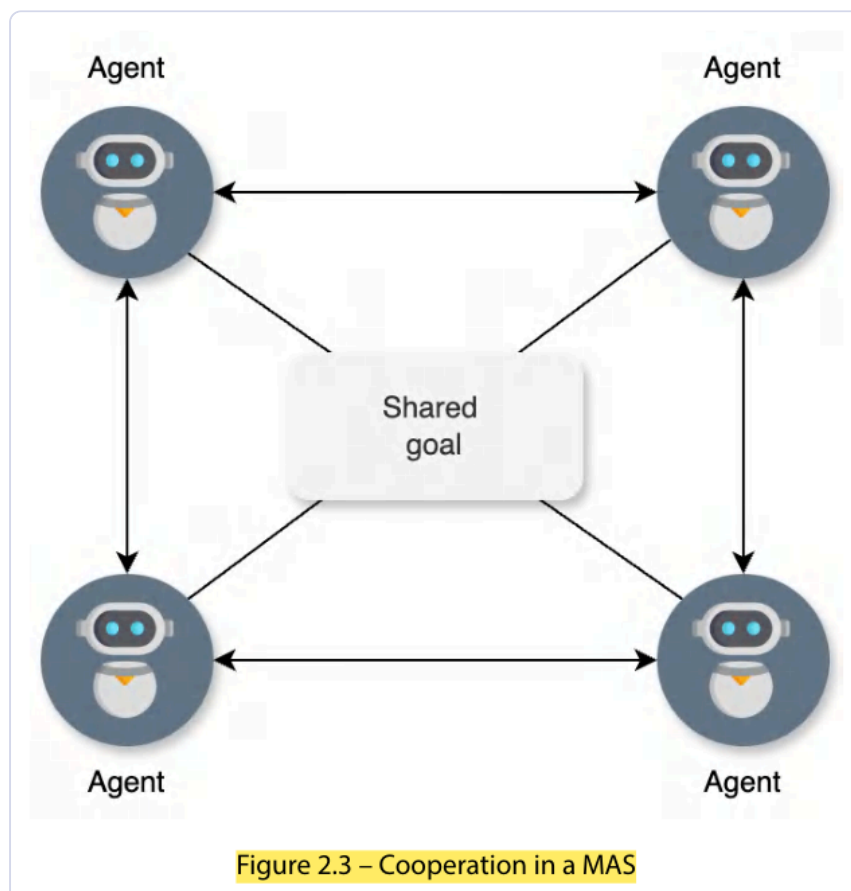


Figure 2.3 — Cooperation in a MAS (all agents link to a shared goal)

(b) Coordination

Manages **interdependencies** from agents' actions — needed when agents share resources, have overlapping responsibilities or conflicting actions. Mechanisms: **task scheduling, resource-allocation management, conflict resolution**. *Example:* manufacturing robots on production lines coordinate shared resources, prevent interference, keep efficiency.

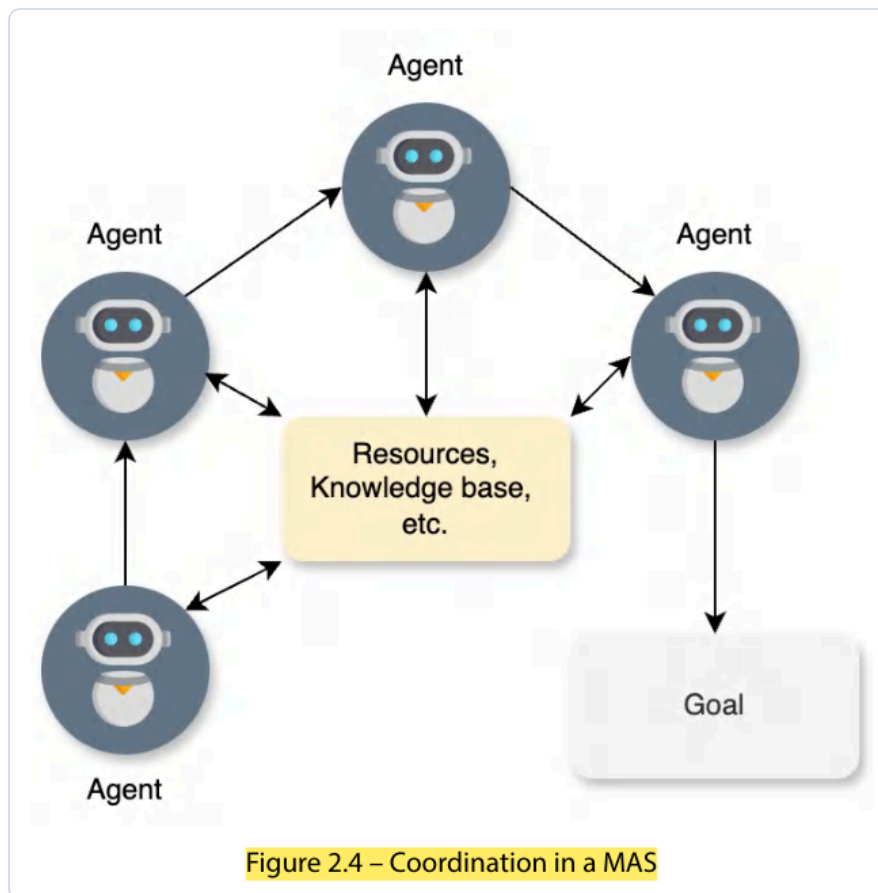


Figure 2.4 — Coordination in a MAS (agents share resources / knowledge base)

(c) Negotiation

Process by which agents reach **agreements** on sharing resources, dividing tasks or resolving conflicts, through **offers, counteroffers and compromises**, even when interests initially conflict. Techniques: **auctions, voting protocols, bargaining strategies, game-theoretic approaches**.

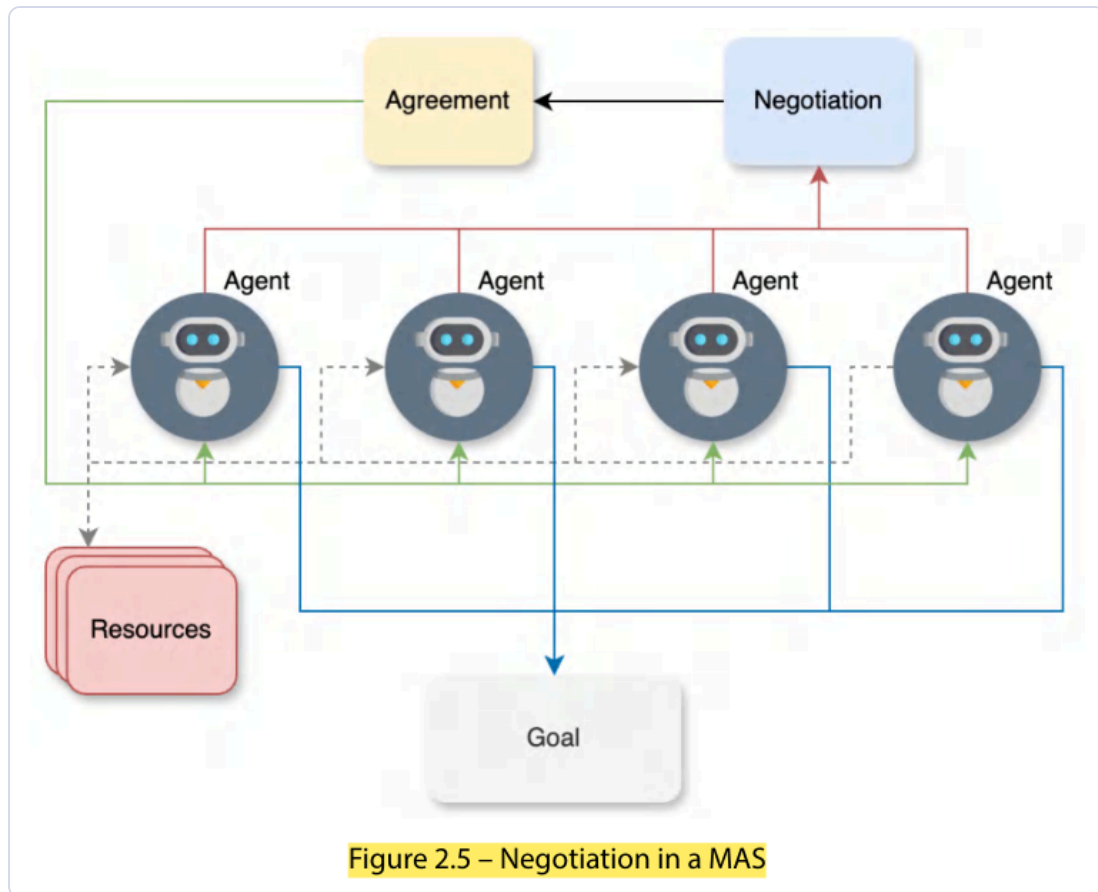


Figure 2.5 — Negotiation in a MAS (agreement via negotiation over resources toward a goal)

Mechanism	Purpose	Travel-industry illustration
Cooperation	Common goal	All agents work to book the traveller's itinerary
Coordination	Manage interdependencies	Travel agency agent needs flight + hotel input to build a package
Negotiation	Reach mutually beneficial agreements	Airline/hotel agents negotiate schedules, rooms, pricing

3.3 MAS in the travel-assistant example — negotiation process

1. **Airline agents** propose available seats, schedules, pricing.
2. **Hotel agents** evaluate proposals using room availability, expected occupancy, demand forecasts and negotiate flight schedules aligning with check-in/out.
3. **Travel-agency agents** negotiate with both, considering customer preferences, budget, travel dates.
4. **Transportation agents** (car rental, shuttles) offer ground transport and pickup/drop-off schedules and costs.

Strategies: auctions, bargaining protocols, game-theory, optimisation; **multi-attribute utility functions** (travel time, cost, comfort, preferences) rank proposals; iterative offers/counteroffers. Decentralised decisions with global optimisation → resilient to changes in demand, supply, pricing.

3.4 Algorithm 2 — MAS for a travel-booking assistant

```
Require: Airline agents A={A1..An}, Hotel agents H={H1..Hm}
Ensure: TravelBookingSystem S with Travel Agency agent TA
function RequestTravelPackage(departure, destination, dates):
  for each Ai in A: available_flights <- Ai.GetAvailableFlights(...)
  for each Hj in H: available_rooms <- Hj.GetAvailableRooms(destination, dates)
  packages <- TA.CompilePackages(available_flights, available_rooms)
  return packages
function BookTravel(selected_package):
  flight_booking <- selected_package.airline.BookFlight()
  room_booking <- selected_package.hotel.BookRoom()
  if both successful: return CreateBooking(flight_booking, room_booking)
  else: return FailureNotification()
function UpdateDynamicPricing(): each Ai.UpdateFlightPrices(); each Hj.UpdateRoomPrices()
while True:
  if NewTravelRequest(): packages <- RequestTravelPackage(...)
    selected <- TA.PresentOptionsToCustomer(packages)
    if selected: booking <- BookTravel(selected); NotifyCustomer(...)
  if TimeToUpdatePricing(): UpdateDynamicPricing()
```

- Defines flight agent, hotel agent and travel-agency agent; agency compiles packages from the best options.
- **Cooperation** — all agents work toward booking the itinerary. **Coordination** — travel agency needs both flight & hotel agents' input.
- Each airline/hotel agent **independently updates its pricing**; main loop handles requests and periodic price updates.
- Still **not intelligent**: needs discrete input (departure/arrival codes) and can't understand text.

4. Enabling Tool Use in Agents

Tool usageThe LLM agent's capability to leverage **external resources/instrumentation** to augment its inherent functionality and decision-making — going beyond an isolated entity that relies only on training data and internal algorithms.

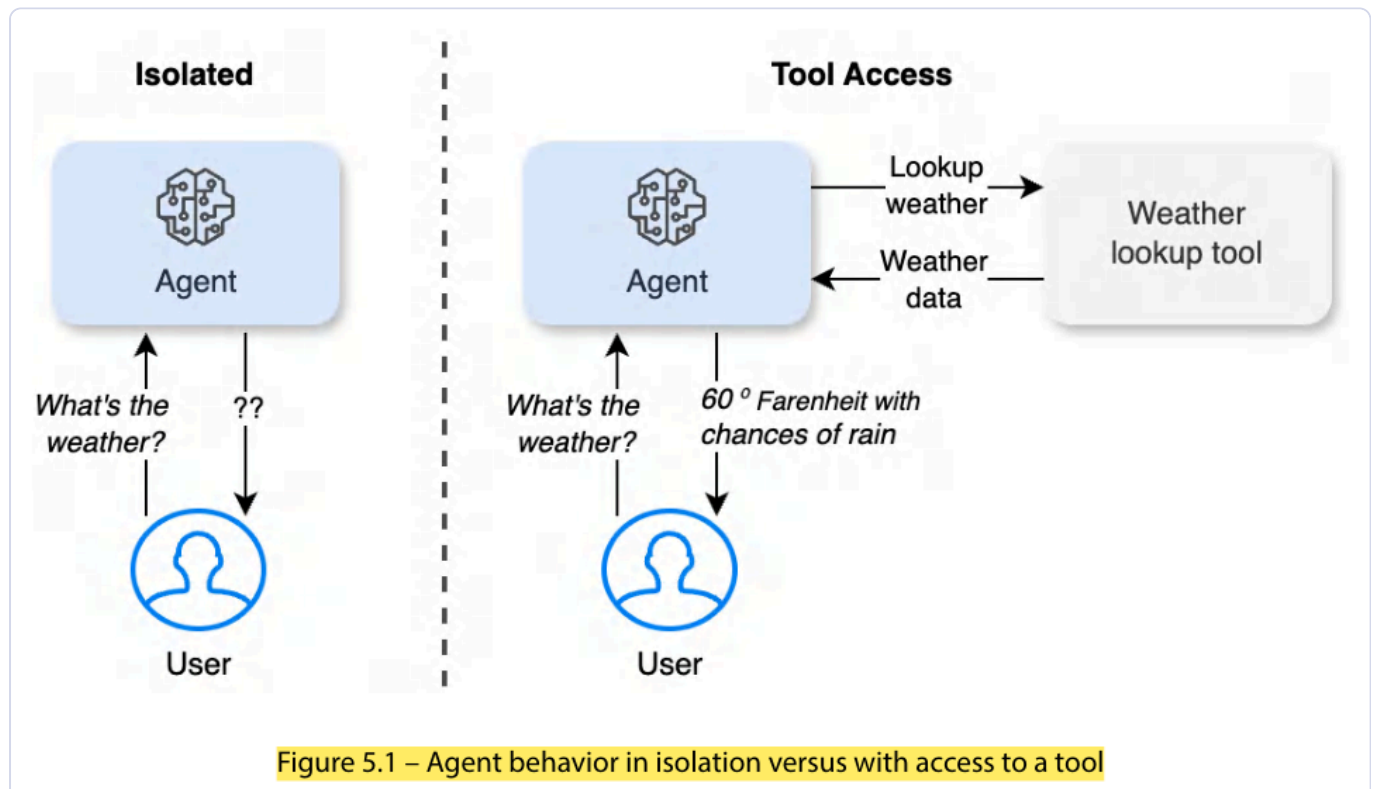


Figure 5.1 — Agent in isolation ("??") vs. agent with a weather lookup tool ("60°F with chances of rain")

- Without a tool the model may **make up an answer** or say it doesn't know the weather (training data has no real-time data); with a real-time tool it answers accurately.
- Tools give **real-time data, specialised tasks, complex workflow management**, broadening competencies to tackle real-world problems; agents **offload** tasks/obtain supplementary data and functions.

4.1 Tool calling vs function calling

Function calling	Tool calling
LLM generates structured calls to predefined functions within the same runtime ; internal tasks (DB lookup, calculation). E.g., fetch a user's profile from a local database.	LLM interacts with external APIs, services or systems for real-time data / specialised tasks. E.g., query a weather API for live updates.

The LLM does NOT execute code it generates a **structured response** stating (1) which tool/function, (2) which parameters, (3) how they must be formatted — like a "detailed instruction" or **dispatcher**. Actual execution is done by an external runtime / **Agent Controller**.

```
{ "function": "weather_lookup",
  "parameters": { "location": "Boston", "date": "10/01/2024" } }
```

```
import requests
def weather_lookup(location: str, date: str) -> dict:
    """A function to lookup weather data that takes location and date as input"""
    API_KEY = "api_key"; base_url = "<api URL>"
    params = {"q": location, "appid": API_KEY, "units": "imperial"} # Fahrenheit
    response = requests.get(base_url, params=params)
    if response.status_code == 200:
        data = response.json()
        return data
```

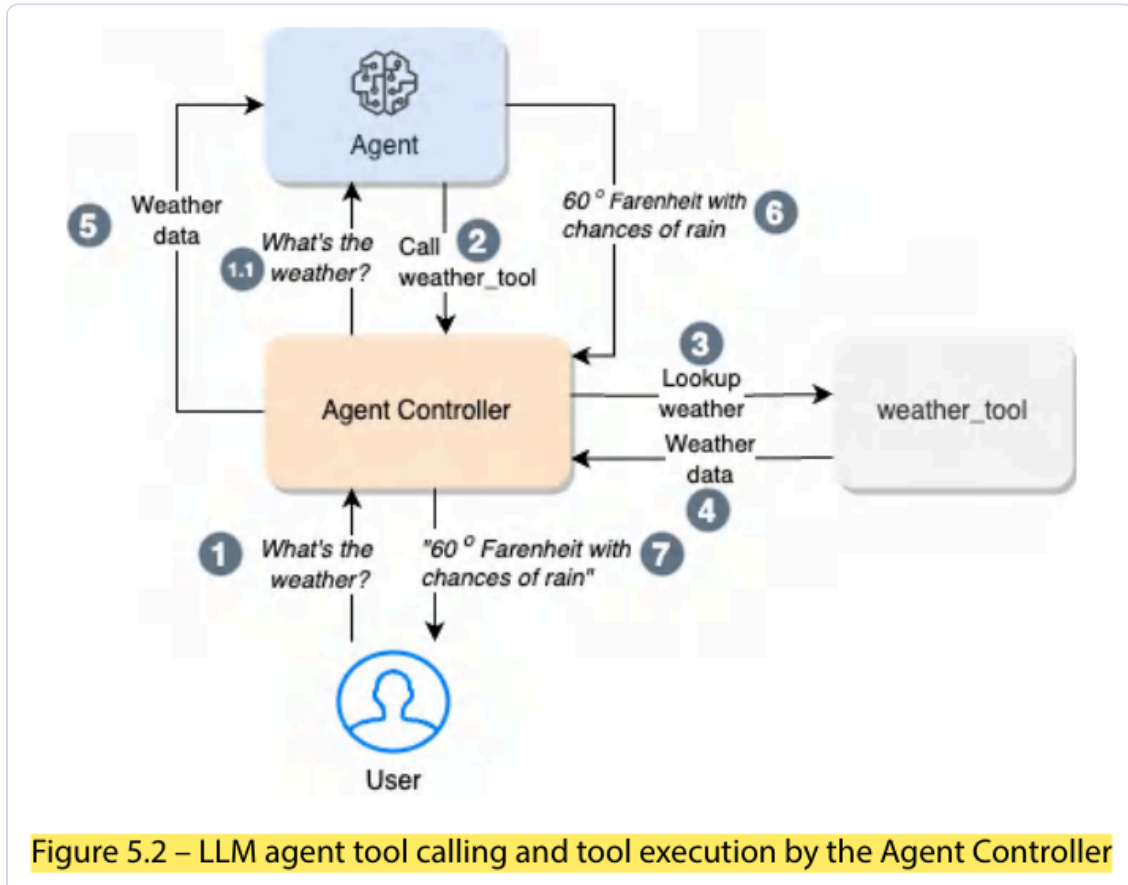


Figure 5.2 – LLM agent tool calling and tool execution by the Agent Controller

Figure 5.2 — LLM agent tool calling and execution by the Agent Controller

- Flow (Fig 5.2):** (1) user asks "What's the weather?" → agent (1.1) decides to **call weather_tool** (2) → Agent Controller (3) **looks up** the tool → (4) weather data returned → (5) data goes to the agent → (6) agent forms the answer ("60°F with chances of rain") → (7) reply to the user.
- At minimum the agent needs the tool's **description** and **expected inputs**; mark parameters **mandatory vs optional**.
 - Not all LLMs are equally accurate at tool calling. Larger models trained explicitly for it: **GPT-4/4o, Claude (Sonnet, Haiku, Opus), Llama 3**; others may work with aggressive prompt engineering.

4.2 Defining tools for agents

Framework approach — docstrings	Direct LLM integration — JSON schema
CrewAI / LangGraph: tools are functions with a docstring (purpose, required & optional arguments, return values). Intuitive; standard programming practice (other languages use different doc conventions).	With raw APIs (Anthropic Claude / OpenAI GPT) define tools as JSON with name, description, input_schema (type, properties, required). Multiple tools = list/array of schemas. Model-dependent — cumbersome with many models, hence the rise of CrewAI, LangGraph, AutoGen for model-agnostic definition.

```
def weather_lookup(location: str, date: str = None):  
    """  
    A tool that can lookup real-time weather data.  
    Arguments:  
        location (str): The location to lookup weather for  
        date (str) Optional: The date in MM/DD/YYYY format  
    """
```

```
{ "name": "weather_lookup",  
  "description": "A tool that can lookup real-time weather data",  
  "input_schema": { "type": "object",  
    "properties": { "location": { "type": "string",  
      "description": "The city and state, e.g. San Francisco, CA" } },  
    "required": ["location"] } }
```

4.3 Types of tools

Tool type	Purpose / examples
APIs	Primary gateway to external services & real-time data; standardised interaction with third-party systems — weather, payments, navigation/maps, flight & hotel booking.
Database tools	Store/retrieve/manage structured or semi-structured data; read & write; persistent info across sessions — customer profiles, transaction history, product catalogs, domain knowledge bases.
Utility functions	Custom local software components for specialised tasks — data processing/analysis, format conversion, maths, NLP; consistent, repeatable operations.
Integration tools	Connect systems & services for workflow automation — calendar sync, document processing, file management, communication systems; orchestrate multi-system workflows.
Hardware interface tools	Bridge digital and physical worlds — IoT devices, robotics, sensor data, physical automation.

4.4 Key considerations when agents use tools

- **Tool composition & chaining** — combine tools into workflows (API flight check → DB preferences → utility itinerary calculation).
- **Tool selection & decision-making** — evaluate context, capabilities, reliability, performance, cost; choose the most efficient when several apply.

- **Error handling & fallbacks** — detect failed API calls, DB connection issues, wrong outputs; fall back to alternative tools/approaches.
- **Tool state management** — initialisation/cleanup, DB connections, API auth tokens, session states.
- **Tool updates & versioning** — version compatibility, deprecated features, new interfaces.
- **Tool security & access control** — authentication credentials, authorisation checks, secure channels, **rate limits and quotas**.

4.5 CrewAI example — tool use for travel

User request "I need flight and hotel options for Rome for 2 adults, June 15–22, 2024, with a total budget of \$3,000."

```
class TravelTools:
    def search_flights(self, ...) -> dict:
        """Basic flight search simulation"""
        return {"flights": [{"airline": "Alitalian airlines", "price": 800, "duration": "9h"}]}
    def check_hotels(self, ...) -> dict:
        """Basic hotel search simulation"""
        return {"hotels": [{"name": "Roma Inn", "price": 150, "rating": 4.0}]}

travel_agent = Agent(role='Travel Agent',
    goal='Find suitable flight and hotel options within budget',
    tools=[TravelTools().search_flights, TravelTools().check_hotels])
search_task = Task(description="Find flights and hotels for 2 adults to Rome, June 15-22, budget $3000",
    agent=travel_agent)
crew = Crew(agents=[travel_agent], tasks=[search_task])
result = crew.kickoff()
```

Concepts shown: tool definition (TravelTools class), agent configuration (role, goal, tools), task specification, tool integration (flight + hotel), execution flow managed by CrewAI. Real implementations would use real APIs/databases and richer data.

4.6 Significance of tools in agentic systems

- Complex problems need a **diverse array of specialised tools**; instead of encapsulating all knowledge in the agent, **leverage appropriate tools as needed** — more efficient & scalable.
- **Healthcare example:** medical databases, clinical decision-support systems, diagnostic algorithms + agent reasoning → accurate, tailored guidance.
- Not limited to software: robotic agents use **physical tools/machinery** (assembly, quality inspection, material handling).
- **Virtual travel agent:** airline/hotel/car-rental/review APIs + travel-advisory databases → personalised recommendations, trade-offs and bookings.
- Effective tool use is a hallmark of truly intelligent agents.

5. Planning Algorithms for Agents — Overview

Planning lets agents reason about actions and devise strategies. An **algorithm** = step-by-step, unambiguous, finite instructions that take inputs and produce an expected output in finite time. For LLM agents, practicality depends on handling **natural language, uncertainty and large state spaces** (all possible conversation states, knowledge contexts and responses — far larger than e.g. robot positions/orientations).

Practicality for LLM agents	Algorithms
Less practical	STRIPS, A*, GraphPlan, MCTS — rigid structure, difficulty with natural language
Moderately practical	Fast Forward (FF) — needs significant adaptation
Most practical	LLM-based planning and HTN (hierarchical task network) — align with how LMs process/decompose tasks (<i>covered in Unit 3</i>)

5.1 Less practical algorithms

Algorithm	Idea	Why impractical for LLM agents
STRIPS (Stanford Research Institute Problem Solver)	States and actions defined by logical predicates ; clear binary (true/false) conditions.	Natural language can't be reduced to true/false; can't model nuanced states ("partially understanding a concept", "somewhat satisfied") — too rigid.
A* planning	Pathfinding using action cost + heuristic estimate of remaining cost.	Defining costs/"distance" between conversation states is highly problematic; mathematical requirements don't fit language.
GraphPlan	Builds a layered graph of actions & effects per time step.	Language interactions don't fit discrete layers with clear cause-effect; combinatorial explosion; hard to find mutual-exclusion relationships → computationally intractable.
MCTS (Monte Carlo Tree Search)	Random simulations — strong in game-like scenarios.	Each simulation needs real LLM calls (prohibitively expensive); vast language space makes random sampling inefficient.

5.2 Moderately practical — Fast Forward (FF)

FF uses **heuristic search with a relaxed (simplified) version of the planning problem** to guide search. Its goal-oriented approach aligns with LLM task completion, and its relaxed plans approximate complex language tasks; heuristic guidance manages huge search spaces; flexible with partial state descriptions. **Challenges:** numeric heuristics don't translate to language states; relaxed plans may oversimplify; defining **delete effects** (what is removed/changed in the conversation state) is hard; state representation needs adaptation. (*Full discussion + code in Unit 3.*)

6. Q&A (from the chapters)

- Q1. Key characteristics of intelligent agents?** Reactivity, proactiveness, social ability, autonomy, and the ability to learn & adapt.
- Q2. Main architectural patterns?** Deliberative (knowledge-based), reactive (behaviour-based), hybrid.
- Q3. Deliberative vs reactive?** Deliberative excels at complex reasoning/planning but may lack real-time responsiveness; reactive suits dynamic environments needing rapid response but lacks long-term planning and abstract reasoning.
- Q4. What is a MAS?** Multiple intelligent agents interacting, cooperating, coordinating toward collective goals — autonomy, interaction, adaptability, distributed control, scalability, heterogeneity, decentralised knowledge.
- Q5. Interaction mechanisms?** Cooperation (common goal), coordination (interdependencies), negotiation (agreements).
- Q6. MAS domains?** Supply chain, traffic control, robotics, environmental monitoring, sensor networks, virtual environments.

7. Unit Summary

- Intelligent agents: reactive, proactive, social, learning, reasoning/planning, autonomous.
- Architectures: **deliberative** (sense-plan-act, symbolic KB), **reactive** (stimulus-response), **hybrid** (reactive + deliberative layers; task decomposition, multiplan selection, external modules, reflection, memory-augmented planning).
- MAS: autonomy, interaction, adaptability, distributed control, scalability, heterogeneity, decentralised knowledge; interactions = cooperation, coordination, negotiation.
- Tools extend LLM agents beyond training data. The LLM only **emits a structured call**; the **Agent Controller executes**. Tools defined via docstrings (frameworks) or JSON schema (APIs).
- Tool types: APIs, databases, utility functions, integration tools, hardware interface tools; considerations: chaining, selection, error handling, state, versioning, security.
- Planning algorithms: STRIPS, A*, GraphPlan, MCTS (less practical); FF (moderate); LLM-based & HTN (most practical).

Likely exam questions 1) Compare deliberative, reactive and hybrid architectures with diagrams. 2) Explain MAS characteristics and interaction mechanisms with the travel example. 3) Differentiate tool calling and function calling; show how an Agent Controller executes a tool call. 4) List and explain types of tools and considerations in tool use. 5) Why are STRIPS/A*/GraphPlan/MCTS impractical for LLM agents?

Sources: Unit-2 Part-1 (Principles of Agentic Systems, Ch.2) & Unit-2 Part-2 (Enabling Tool Use and Planning in Agents, Ch.5) — summarised for RVCE study use.